เอกสารประกอบการสอน: AI3453 Deep Learning

สัปดาห์ที่ 4: Generalization & Regularization

หัวข้อ: การทำให้โมเดล "เก่งจริง" ไม่ใช่แค่ "ท่องจำข้อสอบ"

ผู้สอน: Dr. Sila Temsiririrkkul

คณะวิทยาศาสตร์และเทคโนโลยี มหาวิทยาลัยหัวเฉียวเฉลิมพระเกียรติ

---

1. บทนำ: ปัญหาของการเรียนรู้ (The Learning Problem)

ในสัปดาห์ที่ผ่านมา เราได้เรียนรู้เทคนิคการ "Optimization" หรือการปรับจูนค่า Weight และ Bias เพื่อให้ค่าความผิดพลาด (Loss) ของโมเดลลดต่ำลงที่สุดเท่าที่จะเป็นไปได้ เราเปรียบเสมือนกำลังฝึกฝนให้นักกีฬาวิ่งเข้าเส้นชัยให้เร็วที่สุด แต่คำถามสำคัญที่เกิดขึ้นในสัปดาห์นี้คือ **"การที่โมเดลทำคะแนนได้ดีเยี่ยมในตอนฝึกซ้อม (Training) การันตีได้จริงหรือว่าโมเดลนั้นจะเก่งจริงเมื่อนำไปใช้งาน?"**

หัวใจสำคัญของ Machine Learning ไม่ใช่การทำให้โมเดลจดจำข้อมูลที่เราป้อนให้ได้ครบทุกตัวอักษร เพราะนั่นไม่ต่างอะไรกับการเขียนโปรแกรม Database เพื่อค้นหาข้อมูล แต่เป้าหมายที่แท้จริงของเราคือ **"Generalization"** หรือความสามารถในการนำสิ่งที่เรียนรู้ไปประยุกต์ใช้กับสถานการณ์ใหม่ๆ ที่ไม่เคยเจอมาก่อนได้

ลองจินตนาการถึงนักศึกษาสองคนในการเตรียมตัวสอบวิชาคณิตศาสตร์:

- **นักศึกษา A (Generalization):** พยายามทำความเข้าใจสูตรและหลักการเบื้องหลัง เมื่อเจอโจทย์พลิกแพลงหรือเปลี่ยนตัวเลข เขาก็ยังสามารถแก้โจทย์ได้
- **นักศึกษา B (Overfitting):** ใช้วิธีท่องจำคำตอบของข้อสอบเก่า 10 ปีย้อนหลังได้ทุกข้อ จนได้คะแนนฝึกซ้อมเต็ม 100% แต่เขาไม่ได้เข้าใจหลักการจริงๆ เมื่อเจอข้อสอบจริงที่เปลี่ยนตัวเลขเพียงเล็กน้อย เขากลับทำไม่ได้เลย

ในทาง Deep Learning เรามักจะเจอปัญหาแบบนักศึกษา B บ่อยครั้ง คือโมเดลพยายาม "ท่องจำ" ข้อมูล Training Set จนค่า Loss เข้าใกล้ 0 แต่เมื่อเรานำโมเดลนั้นไปทดสอบกับข้อมูลใหม่ (Test Set) กลับพบว่าความแม่นยำตกต่ำลงอย่างน่าตกใจ นี่คือสัญญาณเตือนว่าโมเดลของเราไม่ได้เกิดการ "เรียนรู้" (Learning) อย่างแท้จริง แต่เป็นเพียงการ "จดจำ" (Memorization) เท่านั้น

ดังนั้น ในบทนี้เราจะมาเรียนรู้วิธีการตรวจสอบว่าโมเดลของเรากำลังเป็นนักศึกษาแบบไหน และเราจะมีวิธีการใดบ้างที่จะดัดนิสัยให้โมเดลเลิกท่องจำและหันมาทำความเข้าใจข้อมูลอย่างแท้จริง

2. เจาะลึกการวินิจฉัยโมเดล (Model Diagnostics Deep Dive)

หัวใจของการวินิจฉัยคือการเข้าใจความสัมพันธ์ระหว่าง **Model Complexity (ความซับซ้อนของโมเดล)** กับ **Error** ซึ่งเราเรียกว่า **Bias-Variance Tradeoff**

2.1 ความหมายเชิงลึกของ Bias และ Variance

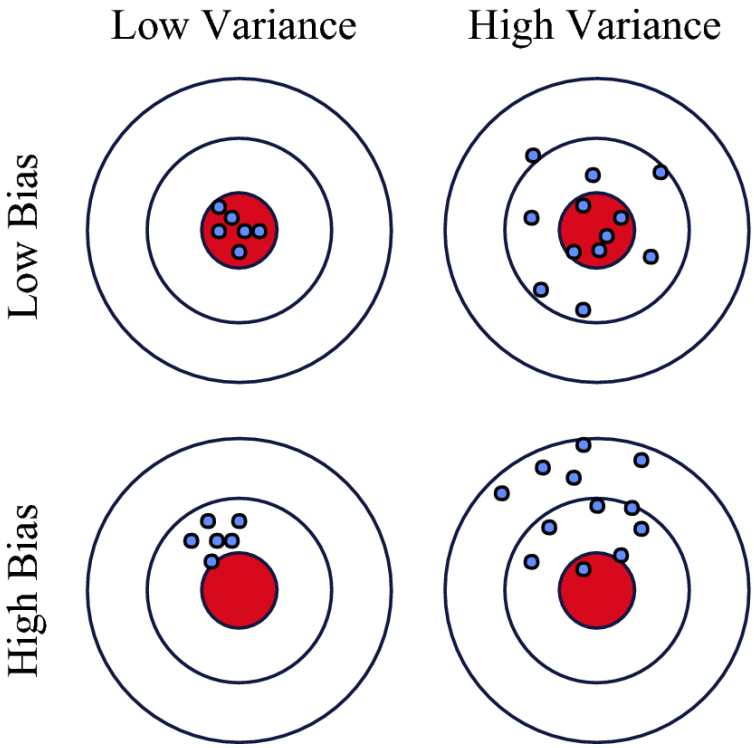
- **Bias (อคติ):** คือความผิดพลาดที่เกิดจาก **"สมมติฐานที่ง่ายเกินไป"** ของโมเดล
    - *ในทางคณิตศาสตร์:* คือระยะห่างระหว่าง "ค่าเฉลี่ยของคำตอบที่โมเดลทาย" กับ "ค่าความจริง"
    - *High Bias:* โมเดลมองโลกแคบ เห็นข้อมูลที่มีความโค้งเว้าซับซ้อน แต่ดันไปเหมาว่าเป็นเส้นตรง (Linear) ทำให้ทายผิดอย่างเป็นระบบ (Systematic Error)
- **Variance (ความแปรปรวน):** คือความผิดพลาดที่เกิดจาก **"ความอ่อนไหวต่อข้อมูลกลุ่มตัวอย่าง"**
    - *ในทางคณิตศาสตร์:* คือการกระจายตัวของผลลัพธ์โมเดล หากเราเปลี่ยน Training Set ไปเป็นกลุ่มอื่นเล็กน้อย ผลลัพธ์จะเปลี่ยนไปคนละทิศละทางหรือไม่?
    - *High Variance:* โมเดลไม่มีจุดยืน พยายามเอาใจข้อมูลทุกจุด รวมถึงจุดที่เป็น Noise ทำให้เส้นกราฟยืกยือ เปลี่ยนไปตามข้อมูลที่ป้อนให้อย่างรุนแรง

2.2 พฤติกรรมของกราฟ Error (The U-Shape Curve)

เมื่อเราเพิ่มความซับซ้อนของโมเดล (เช่น เพิ่มจำนวน Layer หรือ Node):

1. **Training Error:** จะลดลงเรื่อยๆ จนเกือบเป็น 0 (เพราะโมเดลจำได้หมด)
2. **Validation Error:** จะลดลงในช่วงแรก (Underfitting → Optimal) แต่พอเลยจุดสมดุลไป มันจะเริ่ม **"ดีดกลับขึ้น"** (Overfitting) กลายเป็นรูปตัว **U**
3. **The Sweet Spot:** เป้าหมายของเราคือจุดต่ำสุดของรูปตัว U นี้ ซึ่งเป็นจุดที่ Total Error (Bias + Variance) ต่ำที่สุด

2.3 วิเคราะห์อาการป่วยของโมเดล (Model Diagnostics: Bias-Variance Tradeoff)



เมื่อโมเดลทำงานได้ไม่ดี เราจำเป็นต้องวินิจฉัยสาเหตุให้ถูกต้องเหมือนกับหมอวินิจฉัยโรค โดยเครื่องมือหลักที่เราใช้คือแนวคิด **Bias-Variance Tradeoff** ซึ่งแบ่งอาการป่วยออกเป็น 2 ประเภทใหญ่ๆ ดังนี้:

1. Underfitting (High Bias): อาการ "รู้ไม่จริง" หรือ "ดื้อด้าน"

อาการนี้เกิดขึ้นเมื่อโมเดลมีความซับซ้อนน้อยเกินไป จนไม่สามารถจับรูปแบบความสัมพันธ์ (Pattern) ของข้อมูลได้

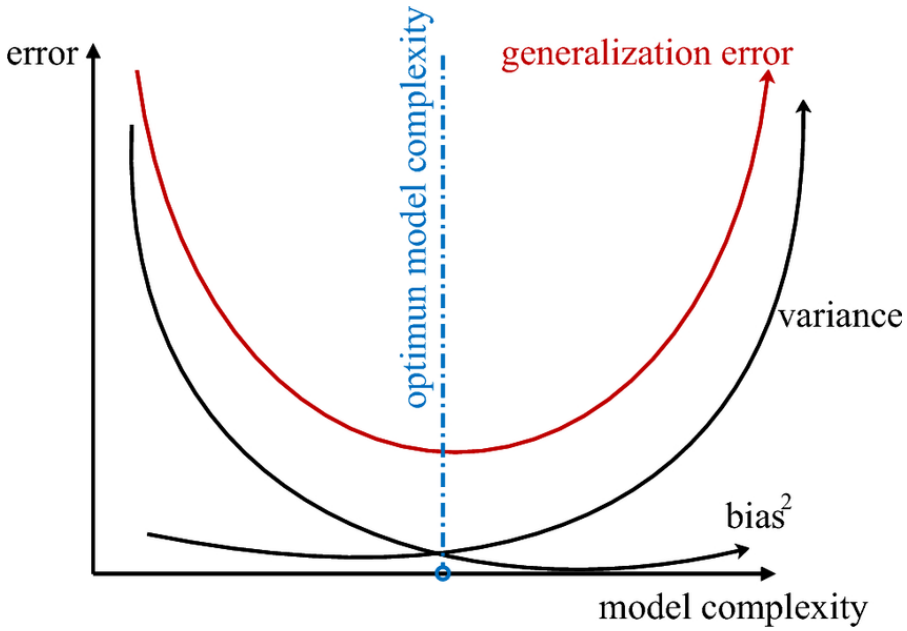
- **คำว่า "High Bias" หมายถึงอะไร?** มันคือการที่โมเดลมี "อคติ" หรือความเชื่อฝังหัวของตัวเองสูงมาก เช่น โมเดลเชื่อว่าข้อมูลต้องเป็นเส้นตรง (Linear) เท่านั้น แม้ว่าข้อมูลจริงจะมีความโค้งมนซับซ้อนแค่ไหน โมเดลก็จะดื้อดึงลากเส้นตรงผ่านข้อมูลเหล่านั้น ทำให้เกิดความผิดพลาดสูง
- **ลักษณะอาการ:** เราจะสังเกตได้ว่าทั้งค่า Training Loss และ Validation Loss จะ **"สูงทั้งคู่"** และไม่ยอมลดลงไม่ว่าจะเทรนไปนานแค่ไหน เปรียบเหมือนนักศึกษาที่ขี้เกียจอ่านหนังสือ หรือพื้นฐานไม่แน่นพอ ทำให้ทำข้อสอบไม่ได้ทั้งตอนซ้อมและตอนสอบจริง
- **ทางแก้:** เราต้องเพิ่ม "ความสามารถ" ให้โมเดล เช่น เพิ่มจำนวน Neuron, เพิ่มจำนวน Layer หรือเปลี่ยน Architecture ให้ซับซ้อนขึ้น เพื่อให้โมเดลมีความยืดหยุ่นในการเรียนรู้มากขึ้น

2. Overfitting (High Variance): อาการ "จำข้อสอบ" หรือ "บ้าจี้ตามข้อมูล"

นี่คือปัญหาที่พบบ่อยที่สุดใน Deep Learning และเป็นหัวข้อหลักที่เราจะแก้กันในสัปดาห์นี้

- **คำว่า "High Variance" หมายถึงอะไร?** มันคือการที่โมเดลมีความ "อ่อนไหว" หรือแปรปรวนไปตามข้อมูลมากเกินไป โมเดลพยายามจะลากเส้นผ่านจุดข้อมูลทุกจุด รวมถึงจุดที่เป็น Noise หรือความผิดพลาดเล็กๆ น้อยๆ ในข้อมูล Training Set ด้วย

- **ลักษณะอาการ:** กราฟจะแสดงให้เห็นชัดเจนว่า **Training Loss ลดต่ำลงมาก (เก่งในบ้าน)** แต่ **Validation Loss กลับมีค่าสูง หรือลดลงแล้วเด้งกลับขึ้นไป (ตกม้าตายเมื่ออยู่นอกบ้าน)** ช่องว่าง (Gap) ระหว่างเส้นกราฟสองเส้นนี้ยิ่งกว้างเท่าไหร่ แสดงว่าอาการ Overfitting ยิ่งรุนแรงเท่านั้น
- **เปรียบเทียบ:** เหมือนคนที่มองก้อนเมฆแล้วจินตนาการเป็นรูปต่างๆ มากมายเกินจริง (เห็น Pattern ในสิ่งที่ไม่มี Pattern) พอไปเจอข้อมูลใหม่ที่ไม่เหมือนเดิม ก็จะทำนายผิดพลาดมหาศาล



3. ยุทธวิธีการแบ่งข้อมูล (Data Splitting Strategy)

ในโลกของ AI กฎเหล็กข้อสำคัญที่สุดไม่ใช่เรื่องของอัลกอริทึมที่ซับซ้อน แต่คือ **"ความซื่อสัตย์ในการประเมินผล"** เพื่อให้มั่นใจว่าโมเดลของเราเก่งจริง ไม่ใช่แค่ฟลุ๊ค เราจำเป็นต้องมีกลยุทธ์การแบ่งข้อมูลที่ถูกต้องตามหลักสถิติ

3.1 กฎเหล็ก 3 กองทัพ (The Three-Fold Split)

เราต้องแบ่งข้อมูลออกเป็น 3 กองเสมอ ห้ามขาดกองใดกองหนึ่ง:

1. **Training Set (60-80%): "แบบฝึกหัดในห้องเรียน"**
    - **หน้าที่:** ใช้สำหรับเทรนโมเดล (ปรับค่า Weight และ Bias)
    - โมเดลจะ **"เห็นและเรียนรู้"** จากข้อมูลกองนี้โดยตรง
2. **Validation Set (10-20%): "การสอบย่อย (Quiz)"**
    - **หน้าที่:** ใช้สำหรับวัดผลระหว่างเทรนเพื่อปรับจูน Hyperparameters (เช่น Learning Rate, จำนวน Layer) และตัดสินใจว่าเมื่อไหร่ควรหยุดเทรน (Early Stopping)
    - โมเดล **"ถูกทดสอบ"** แต่ **"ไม่ได้เรียนรู้"** (Weight ไม่เปลี่ยนจากกองนี้)
3. **Test Set (10-20%): "การสอบปลายภาค (Final Exam)"**
    - **หน้าที่:** ใช้สำหรับวัดประสิทธิภาพครั้งสุดท้ายเมื่อจบโปรเจกต์
    - **ข้อห้าม:** เก็บใส่ตู้เซฟ! ห้ามนำมาเทรนหรือจูนค่าใดๆ เด็ดขาด เพื่อป้องกัน **Data Leakage (การรั่วไหลของข้อมูล)**

3.2 วิธีปฏิบัติจริง: การหั่น 2 จังหวะ (The Nested Split)

ในทางปฏิบัติ (โดยเฉพาะเมื่อใช้ Scikit-Learn) เรามักไม่สามารถแบ่ง 3 กองได้ในคำสั่งเดียว เราจึงใช้วิธี **"แบ่งซ้อน" (Nested Split)** ดังนี้:

- **จังหวะที่ 1:** แยกข้อมูลทั้งหมดออกเป็น 2 กองก่อน คือ **Hold-out Test Set (20%)** และข้อมูลที่เหลือ (Remaining 80%)
- **จังหวะที่ 2:** นำข้อมูลที่เหลือ (Remaining 80%) มาแบ่งอีกรอบเป็น **Train** และ **Validation** (เช่น แบ่ง 80:20 ของก้อนที่เหลือ)

ตัวอย่างโค้ดมาตรฐาน:

Python

```
# 1. แยก Test Set ออกไปก่อน (เก็บเข้าตู้เซฟ)
X_rem, X_test, y_rem, y_test = train_test_split(X, y, test_size=0.2)

# 2. นำส่วนที่เหลือ (Remaining) มาแบ่งเป็น Train และ Val
# (0.25 ของ 80% คือ 20% ของข้อมูลตั้งต้นพอดี)
X_train, X_val, y_train, y_val = train_test_split(X_rem, y_rem, test_size=0.25)
```

*ผลลัพธ์ที่ได้จะเป็นสัดส่วนประมาณ **60% : 20% : 20%** ซึ่งเป็นสัดส่วนมาตรฐานที่ยอมรับกันทั่วโลก*

3.3 ทางเลือกสำหรับข้อมูลน้อย: Cross-Validation (K-Fold)

ในกรณีที่เรามีข้อมูลน้อยมาก (Small Data) เช่น มีภาพแค่ 100-200 ภาพ การแบ่ง Test Set ออกไป 20 ภาพอาจทำให้เกิดความเสี่ยง เพราะ 20 ภาพนั้นอาจจะเป็นภาพที่ง่ายเกินไปหรือยากเกินไป (Bias) จนวัดผลไม่ได้มาตรฐาน

วิธีแก้: ใช้เทคนิค **K-Fold Cross-Validation** (การหมุนเวียนสอบ)

- **หลักการ:** แบ่งข้อมูลเป็น K ส่วน (เช่น 5 ส่วน) แล้ววนลูปเทรน 5 รอบ โดยผลัดกันให้แต่ละส่วนทำหน้าที่เป็น Validation Set
- **ผลลัพธ์:** นำคะแนนความแม่นยำจากทั้ง 5 รอบมาหาค่าเฉลี่ย
- **ข้อดี:** ข้อมูลทุกตัวถูกใช้คุ้มค่า ทั้งเป็นตัวเรียนและตัวสอบ มั่นใจได้ว่าโมเดลเก่งจริง
- **ข้อเสีย:** ใช้เวลาเทรนนานขึ้น K เท่า

3.4 มุมมองสถิติในยุค Big Data: ทำไม Deep Learning ถึงแบ่งแค่ครั้งเดียว?

นักศึกษาอาจสงสัยว่า *"ทำไม Google หรือ Facebook เทรน AI ถึงไม่ทำ Cross-Validation? ทำไมเขาแบ่ง Train/Test แค่รอบเดียวจบ?"*

คำตอบอยู่ที่ **กฎของจำนวนมหาศาล (Law of Large Numbers)**:

- **ทฤษฎี:** เมื่อจำนวนข้อมูล (N) มีมหาศาล (หลักแสนหรือล้าน) การสุ่มตัวอย่างออกมาเพียงส่วนเล็กน้อย (เช่น 10,000 รูป) ก็เพียงพอที่จะเป็น **"ตัวแทนที่สมบูรณ์ของประชากร" (Representative Sample)** ได้แล้ว
- **ในทางปฏิบัติ:** ถ้าเรามีข้อมูล Big Data ข้อมูลใน Train Set, Validation Set และ Test Set จะมีหน้าตาการกระจายตัว (Distribution) ที่เหมือนกันเป๊ะโดยอัตโนมัติ ทำให้การแบ่งแค่รอบเดียว (Single Hold-out) ให้ผลที่แม่นยำเพียงพอและเชื่อถือได้ทางสถิติ ไม่จำเป็นต้องเสียเวลาทำ Cross-Validation หลายรอบครับ

สรุปคำแนะนำ (Rule of Thumb):

- **ข้อมูลน้อย (< 10,000):** ควรใช้ **Cross-Validation** เพื่อความชัวร์
- **ข้อมูลเยอะ (> 100,000):** ใช้ **Standard Split (แบ่ง 3 กอง)** ได้เลย ประหยัดเวลาและแม่นยำพอ

---

4. เทคนิค Regularization: ยารักษาอาการ Overfitting จากงานวิจัยสู่การปฏิบัติ

เมื่อโมเดล Deep Learning มีความซับซ้อนสูงจนเกิดอาการ Overfitting หรือการจดจำข้อสอบแทนที่จะเรียนรู้หลักการ เราจำเป็นต้องนำกระบวนการที่เรียกว่า **Regularization** เข้ามาช่วยเพื่อจำกัดขอบเขตการเรียนรู้ของโมเดลให้อยู่ในร่องในรอย เทคนิคเหล่านี้ไม่ได้เกิดขึ้นมาลอยๆ แต่ผ่านการพิสูจน์ทางคณิตศาสตร์และได้รับการยอมรับในระดับสากล โดยมี 3 เทคนิคสำคัญที่ถือเป็นมาตรฐานในวงการ ดังนี้

4.1 L2 Regularization (Weight Decay): ศิลปะแห่งความเรียบง่าย

หนึ่งในแนวคิดพื้นฐานที่สุดในการควบคุมความซับซ้อนของโมเดลถูกนำเสนอในงานวิจัยของ **Krogh และ Hertz (1992)** เรื่อง *"A Simple Weight Decay Can Improve Generalization"* ซึ่งนำหลักการปรัชญาเก่าแก่อย่าง **Occam's Razor** มาใช้ในทางคณิตศาสตร์ โดยหลักการนี้เชื่อว่า "หากมีคำอธิบายสองแบบที่ให้ผลลัพธ์เหมือนกัน คำอธิบายที่เรียบง่ายกว่ามักจะเป็นคำตอบที่ถูกต้องเสมอ"

ในมุมมองของ Neural Network ความ "ซับซ้อน" มักจะสะท้อนออกมาผ่านค่า Weight ที่มีขนาดใหญ่ผิดปกติ โมเดลที่ Overfit มักจะพยายามขยายค่า Weight ให้สูงมากเพื่อบิดเส้นกราฟให้คดเคี้ยวไปหาจุดข้อมูลที่เป็น Noise การแก้ปัญหานี้ทำได้โดยการเพิ่ม "บทลงโทษ" (Penalty Term) เข้าไปในสมการ Loss Function ซึ่งเราเรียกว่าเทอม **L2 Norm** หรือ $\frac{\lambda}{2} \|w\|^2$

กลไกนี้จะบีบให้ Optimizer ต้องทำงานหนักขึ้น 2 เท่า คือนอกจากจะต้องลดค่า Error จากการทำนายแล้ว ยังต้องคอยพยายามกดค่า Weight ให้น้อยลงที่สุดเท่าที่จะทำได้ ผลลัพธ์ที่ได้คือโมเดลจะเรียนรู้ที่จะกระจายความสำคัญไปยัง Weight หลายๆ ตัวอย่างสมดุล แทนที่จะพึ่งพา Weight ตัวใดตัวหนึ่งมากเกินไป ทำให้เส้นแบ่งการตัดสินใจ (Decision Boundary) มีความเรียบเนียนและเสถียรขึ้น ลดความอ่อนไหวต่อข้อมูลรบกวนได้อย่างมีนัยสำคัญ

1. สมการหน้าตาเป็นอย่างไร? (The Formula)

สมมติเดิมทีเราใช้ Loss ปกติ (เช่น MSE หรือ Cross-Entropy) เราจะเรียกมันว่า $L_{original}$
เมื่อเราใช้ Weight Decay (L2 Regularization) สมการ Loss ใหม่ (L_{total}) จะกลายเป็น:

$$L_{total} = \underbrace{L_{original}}_{\text{ความผิดพลาดจากการทำนาย}} + \underbrace{\frac{\lambda}{2}\sum w^2}_{\text{บทลงโทษ (Penalty)}}$$

---

2. ไขรหัสตัวแปรทีละตัว

ส่วนที่ 1: $\sum w^2$ (Sigma w squared)

- **w คืออะไร?:** ใช่ครับ มันคือค่า **Weight** ทุกตัวในโมเดลครับ (ทั้งใน Layer แรก, Layer กลาง, Layer ท้าย)
- **ทำไมต้องยกกำลังสอง?:** เพื่อให้ค่าเป็นบวกเสมอ (Positive) และเพื่อลงโทษ Weight ที่มีค่า **"มากผิดปกติ"** ให้หนักขึ้น (เช่น Weight = 10 โดนโทษ $10^2 = 100$ แต่ Weight = 1 โดนโทษแค่ $1^2 = 1$)
- **หมายเหตุ:** ปกติเราจะเอา Weight ทุกตัวมารวมกัน แต่ **จะไม่รวมค่า Bias ($\boldsymbol{b}$)** ครับ (เรายอมให้ Bias มีค่าเท่าไหร่ก็ได้ เพื่อให้กราฟขยับขึ้นลงได้อิสระ)

ส่วนที่ 2: $\boldsymbol{\lambda}$ (Lambda)

- **คืออะไร?:** มันคือ **"อัตราโทษปรับ" (Penalty Strength)** หรือ **Regularization Rate** ครับ
- **หน้าที่:** เป็นตัวควบคุมความรุนแรงของการลงโทษครับ เปรียบเหมือน "อัตราภาษีความซับซ้อน"
    - **ถ้า $\boldsymbol{\lambda}$ มาก (เช่น 0.1):** โทษปรับรุนแรงมาก → บีบให้โมเดลต้องลดค่า Weight ลงให้เหลือเล็กจิ๋วมากๆ (โมเดลจะเรียบง่ายสุดๆ จนอาจ Underfit)
    - **ถ้า $\boldsymbol{\lambda}$ น้อย (เช่น 0.00001):** โทษปรับเบาบาง → ปล่อยให้ Weight มีค่ามากได้ตามใจชอบ (แทบไม่ต่างจากเดิม)
    - **ถ้า $\boldsymbol{\lambda} = \mathbf{0}$:** คือไม่ใช้ Regularization เลย (กลับไปเป็นสมการปกติ)

---

3. กลไกการทำงาน: มันลด Weight ได้ยังไง?

ทำไมการบวกเพิ่มใน Loss ถึงทำให้ Weight ลดลง? มาดูตอน **Backpropagation** ครับ

เมื่อเราหาอนุพันธ์ (Gradient) เพื่ออัปเดต Weight:

$$\frac{\partial L_{total}}{\partial w} = \frac{\partial L_{original}}{\partial w} + \lambda w$$

และสูตรอัปเดต Weight คือ:

$$w_{new} = w_{old} - \eta \cdot (\text{Gradient})$$

$$w_{new} = w_{old} - \eta \cdot \left(\frac{\partial L_{original}}{\partial w} + \lambda w_{old} \right)$$

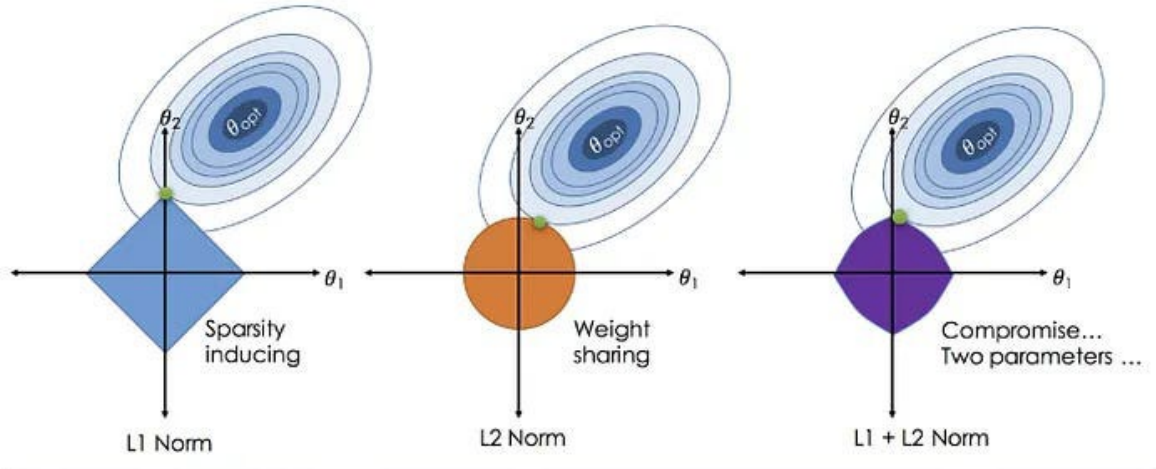
เมื่อจัดรูปสมการใหม่ จะได้ความจริงที่น่าทึ่ง:

$$w_{new} = \underbrace{(1 - \eta\lambda)}_{\text{น้อยกว่า } 1} w_{old} - \eta \frac{\partial L_{original}}{\partial w}$$

ความหมาย:

ในทุกๆ รอบการเทรน ก่อนที่โมเดลจะเรียนรู้จากข้อมูล (เทอมหลัง) มันจะถูก **"คูณด้วยค่าที่น้อยกว่า 1"** (เช่น 0.99)

นั่นแปลว่า **"ค่า Weight จะค่อยๆ หดสั้นลง (Decay) ไปเรื่อยๆ ในทุก Step"** ครับ



คำอธิบายภาพ: เรขาคณิตของ Regularization (The Geometry of Regularization)

ภาพนี้แสดงการหาคำตอบของโมเดลภายใต้ "ข้อจำกัด" (Constraint) ของ Regularization แต่ละแบบ โดย:

- **วงรีสีฟ้า (Contours):** คือค่า Error (Loss Function) ของโมเดล ยิ่งเข้าใกล้จุดศูนย์กลาง (θ_{opt}) Error ยิ่งต่ำ
- **พื้นที่รูปทรงเรขาคณิต (Constraint Regions):** คือขอบเขตที่ Regularization "อนุญาต" ให้ค่า Weight (θ_1, θ_2) อยู่ได้
- **จุดสีเขียว:** คือคำตอบสุดท้ายที่โมเดลเลือก (จุดที่วงรี Error มาสัมผัสกับขอบเขต Regularization พอดี)

---

1. ภาพซ้าย: L1 Norm (Lasso Regression)

- **รูปทรง:** สี่เหลี่ยมขนมเปียกปูน (Diamond Shape) เกิดจากสมการ $|\theta_1| + |\theta_2| \leq C$
- **คำอธิบาย:**
    - สังเกตว่ารูปทรงนี้มี **"มุมแหลม" (Corners)** ยื่นออกไปตามแนวแกน (θ_1, θ_2)
    - เมื่อวงรี Error ขยายตัวออกมาหาคำตอบ มันมีโอกาสสูงมากที่จะมา "ชน" ที่ **มุมแหลม** พอดี (ตามจุดสีเขียวในภาพ)
    - **ผลลัพธ์:** การที่จุดสัมผัสอยู่บนแกนพอดี แปลว่าค่า Weight อีกตัวหนึ่งมีค่าเป็น **0** (ในภาพคือ $\theta_1 = 0$)

- **สรุป (Key Takeaway):** นี่คือเหตุผลทางเรขาคณิตว่าทำไม L1 ถึงเรียกว่า **"Sparsity Inducing"** คือมันช่วย **"ตัดตัวแปร" (Feature Selection)** ที่ไม่จำเป็นทิ้งไปเลย (ให้ค่า Weight เป็น 0)

2. ภาพกลาง: L2 Norm (Ridge Regression)

- **รูปทรง:** วงกลม (Circle) เกิดจากสมการ $\theta_1^2 + \theta_2^2 \leq C$
- **คำอธิบาย:**
    - รูปวงกลมมีความโค้งมนสม่ำเสมอ ไม่มีมุมแหลมที่ยื่นออกไป
    - เมื่อวงรี Error มาสัมผัส ส่วนใหญ่มักจะชนที่ **ขอบโค้ง** (จุดสีเขียว) ซึ่งไม่ใช่จุดตัดแกน
    - **ผลลัพธ์:** ค่า Weight (θ_1, θ_2) จะถูกบีบให้มีค่า **"น้อยลง"** (Shrinkage) เข้าใกล้ศูนย์ แต่ **"ไม่เป็น 0 สนิท"**
- **สรุป (Key Takeaway):** L2 ช่วยลดความซับซ้อนของโมเดลโดยการเกลี่ยค่า Weight ให้น้อยลง (Weight Decay) แต่ยังเก็บ Feature ทุกตัวไว้ เหมาะสำหรับการป้องกัน Overfitting โดยไม่ทิ้งข้อมูล

3. ภาพขวา: Elastic Net (L1 + L2 Norm)

- **รูปทรง:** ลูกผสม (Hybrid) ระหว่างสี่เหลี่ยมและวงกลม (ขอบโค้งแต่มีมุมแหลม) เกิดจากสมการ

$$\alpha|\theta| + (1-\alpha)\theta^2 \leq C$$

- **คำอธิบาย:**
    - นี่คือทางสายกลาง (Compromise) ที่เอาข้อดีของทั้งคู่มารวมกัน
    - **ส่วนมุมแหลม (จาก L1):** ยังคงมีอยู่ ทำให้โมเดลยังสามารถทำ **Feature Selection** (ตัดตัวแปรเป็น 0) ได้บ้าง
    - **ส่วนขอบโค้ง (จาก L2):** ช่วยให้พื้นที่ยืดหยุ่นขึ้น ทำให้รองรับกรณีที่ตัวแปรมีความสัมพันธ์กันสูง (Correlated Features) ได้ดีกว่า L1 เพียวๆ
- **สรุป (Key Takeaway):** Elastic Net คือการประนีประนอม เหมาะกับโจทย์ที่มีข้อมูลเยอะและมีตัวแปรที่สัมพันธ์กัน (Multicollinearity) โดยเราสามารถจูนได้ว่าจะให้เอียงไปทาง L1 หรือ L2 มากกว่ากัน

L2 Regularization (Ridge Regression): เชือกวงกลมกับการลดทอนที่นุ่มนวล เทคนิคนี้ (มักเรียกใน Deep Learning ว่า Weight Decay) สร้างข้อจำกัดทางเรขาคณิตเป็นรูป **"วงกลม"** (ใน 2 มิติ)

หรือทรงกลม เมื่อโมเดลพยายามวิ่งไปหาจุดต่ำสุดของ Loss Function จนเชือกตึง จุดที่โมเดลจะไปหยุดมักจะเป็นจุดสัมผัสระหว่างขอบโค้งของวงกลมกับเส้นชั้นความสูงของ Error (Contour) ด้วยความที่ขอบของมันมีความโค้งมนสม่ำเสมอ จุดสัมผัสจึงมักจะเกิดขึ้นที่ตำแหน่งใดก็ได้ที่ไม่ใช่แกนหลัก

ในทางปฏิบัติและทฤษฎีตามงานวิจัยของ **Krogh & Hertz (1992)** L2 จะทำหน้าที่เป็น **"Soft Shrinkage"** คือบีบให้ค่า Weight ลดลงเข้าใกล้ 0 อย่างนุ่มนวลตามสัดส่วน (Scaling) แต่จะ **"ไม่ทำให้ Weight เป็น 0 สนิท"** เปรียบเสมือนการเดินเข้าหาประตูทีละก้าวที่สั้นลงเรื่อยๆ เราจะเข้าใกล้ประตูมาก แต่ไม่มีวันเดินชนประตู ดังนั้น L2 จึงเหมาะสำหรับงานที่เราเชื่อว่าตัวแปรทุกตัวมีความสำคัญและต้องการเก็บรักษาข้อมูลทั้งหมดไว้ เพียงแต่ลดความไว (Sensitivity) ของโมเดลต่อสัญญาณรบกวนลง

L1 Regularization (Lasso Regression): เชือกสี่เหลี่ยมกับการตัดตัวแปร ในทางกลับกัน L1 สร้างข้อจำกัดทางเรขาคณิตเป็นรูป **"สี่เหลี่ยมขนมเปียกปูน" (Diamond Shape)** ซึ่งมีความพิเศษตรงที่มี **"มุมแหลม"** ยื่นออกมาตามแนวแกนต่างๆ ในทางสถิติเมื่อวงรีของ Error ขยายตัวออกมาสัมผัสกับพื้นที่ข้อจำกัดนี้ มันมีโอกาสสูงมากที่จะมาชนที่ "มุมแหลม" เหล่านี้ก่อนส่วนอื่น และในทางคณิตศาสตร์ "มุม" คือตำแหน่งที่ค่า Weight ตัวใดตัวหนึ่งเท่ากับ 0 พอดี

ด้วยเหตุนี้ L1 จึงมีคุณสมบัติที่เรียกว่า **Sparsity Inducing** หรือการเป็น **Feature Selector** ในตัว คือมันสามารถ "ตัด" ตัวแปรที่ไม่จำเป็นทิ้งไปได้เลย (Weight = 0) ทำให้โมเดลมีขนาดเล็กลงและตีความง่ายขึ้น แต่ข้อเสียคือหากตัวแปรมีความสัมพันธ์กันสูง (Correlated) L1 มักจะสุ่มเลือกตัวใดตัวหนึ่งเก็บไว้และทิ้งตัวที่เหลือไปอย่างไม่ไยดี

Elastic Net: ทางสายกลางที่สมดุล เพื่อแก้ข้อจำกัดของทั้งคู่ **Elastic Net** จึงถือกำเนิดขึ้นโดยการผสมผสานรูปทรงของ L1 และ L2 เข้าด้วยกัน ทำให้พื้นที่ข้อจำกัดมีลักษณะพิเศษคือ **"มีมุมแหลมแบบ L1 แต่ขอบมีความโค้งมนแบบ L2"** รูปทรงลูกผสมนี้ช่วยให้เราได้ประโยชน์จากทั้งสองโลก คือยังมีความสามารถในการคัดเลือกตัวแปร (ตัดค่าเป็น 0 ที่มุมแหลม) แต่ในขณะเดียวกันก็มีความยืดหยุ่นในการจัดการกับกลุ่มตัวแปรที่สัมพันธ์กันได้ดีขึ้น (Group Effect) ไม่ตัดทิ้งสุ่มสี่สุ่มห้าเหมือน L1 เพียวๆ

4.2 Dropout: พลังของการเรียนรู้แบบทีม (Ensemble Learning)

เทคนิคที่ปฏิวัติวงการ Deep Learning อย่างแท้จริงเกิดขึ้นในปี 2014 เมื่อทีมวิจัยนำโดย **Nitish Srivastava และ Geoffrey Hinton** ตีพิมพ์ผลงานเรื่อง *"Dropout: A Simple Way to Prevent Neural Networks from Overfitting"* ซึ่งนำเสนอแนวคิดที่ดูเหมือนจะขัดกับสามัญสำนึก นั่นคือการ "ทำลาย" โครงข่ายบางส่วนทิ้งในระหว่างการเทรน

Dropout ทำงานโดยการสุ่ม "ปิด" การทำงานของ Neuron บางตัวด้วยความน่าจะเป็นที่กำหนด (เช่น 50%) ในทุกๆ รอบของการเทรน การทำเช่นนี้ช่วยแก้ปัญหาใหญ่ที่เรียกว่า **Co-adaptation** หรือการที่ Neuron แอบ "สมรู้ร่วมคิด" หรือพึ่งพากันเองมากเกินไป เช่น Neuron ตัวหนึ่งอาจจะรอแก้ไขข้อผิดพลาดของเพื่อนข้างๆ จนไม่ได้เรียนรู้ฟีเจอร์ที่เป็นประโยชน์ด้วยตัวเอง เมื่อ Dropout เข้ามาขัดจังหวะ Neuron ที่เหลืออยู่จึงถูกบังคับให้ต้องเก่งขึ้นและทำงานได้ด้วยตัวเอง (Robust Features)

นอกจากนี้ ในเชิงทฤษฎี Hinton ยังอธิบายว่าการใช้ Dropout เปรียบเสมือนการเทรนโมเดลย่อยๆ (Thinned Networks) จำนวนมหาศาล (2^N รูปแบบ) ที่มีโครงสร้างต่างกันแต่แชร์ Weight ร่วมกัน เมื่อถึงเวลาใช้งานจริง (Test Phase) การเปิด Neuron ทุกตัวให้ทำงานพร้อมกันจึงมีค่าเท่ากับนำโมเดลย่อยเหล่านั้นมา "โหวต" หรือหาค่าเฉลี่ยร่วมกัน ซึ่งตรงกับหลักการ **Ensemble Learning** ที่ระบุว่าการรวมพลังของโมเดลหลายตัวย่อมให้ผลลัพธ์ที่แม่นยำกว่าโมเดลเดี่ยวเสมอ

คณิตศาสตร์เบื้องหลัง Dropout (The Mathematics of Dropout)

ในทางปฏิบัติ เรามอง Dropout เป็นการ "ตัด" Neuron ทิ้ง แต่ในทางคณิตศาสตร์ มันคือการคูณ Matrix ด้วย **Bernoulli Random Variable** และการจัดการ **Scaling Factor** เพื่อรักษาสมดุลของพลังงานในระบบ

1. การสุ่มแบบแบร์นูลลี (Bernoulli Masking)

สมมติให้ h คือเวกเตอร์ของผลลัพธ์จาก Hidden Layer (ก่อนจะส่งไปชั้นถัดไป)

ในสภาวะปกติ:

$$y = f(Wx + b)$$

เมื่อเราใส่ Dropout เข้าไป เรากำลังสร้างเวกเตอร์สุ่มตัวใหม่ขึ้นมา ชื่อว่า r (Mask Vector) โดยที่ r มีการแจกแจงแบบ **Bernoulli Distribution**:

- $r_j \sim \text{Bernoulli}(p)$
- หมายความว่า r_j มีโอกาสเป็น **1** ด้วยความน่าจะเป็น p และเป็น **0** ด้วยความน่าจะเป็น $1 - p$

สมการ Forward Pass จะเปลี่ยนไปเป็น:

$$y = f(Wx + b) \odot r$$

(สัญลักษณ์ $\odot$ คือ Element-wise multiplication หรือการคูณกันตัวต่อตัว)

ผลทางคณิตศาสตร์: ค่าในเวกเตอร์ y บางตำแหน่งจะถูก "บังคับให้เป็น 0" (Masked out) ทันที ทำให้ Gradient ที่ไหลย้อนกลับมา (Backprop) ที่ตำแหน่งนั้นเป็น 0 ไปด้วย น้ำหนัก W ที่เชื่อมต่อกับตำแหน่งนั้นจึงไม่ได้รับการอัปเดตในรอบนั้น

---

2. ปัญหา Scaling และการแก้ด้วย Inverted Dropout

นี่คือจุดที่นักศึกษา (และไลบรารีเก่าๆ) มักจะสับสนครับ

- **ตอน Train:** เราเปิด Neuron แค่ p ส่วน (เช่น 50% หรือ $p=0.5$) ดังนั้นค่าผลรวมของ Output ($E[y]$) จะหายไปครึ่งหนึ่ง
- **ตอน Test:** เราเปิด Neuron **ครบทุกตัว (100%)** ทำให้ค่าผลรวม Output มีพลังงานมากเกินไป (Over-excited) เป็น 2 เท่าของตอนเทรน ซึ่งจะทำให้โมเดลพัง

วิธีแก้ทางคณิตศาสตร์ (Inverted Dropout):

แทนที่จะไปปรับตอน Test (ซึ่งยุ่งยาก) เราจะ **"ชดเชยค่า" ตั้งแต่ตอน Train เลย** โดยการหารด้วย p

$$\tilde{y} = \frac{1}{p}(x \odot r)$$

- **ตัวอย่าง:** ถ้า $p = 0.5$ (Drop 50%) เราจะคูณ Output ที่เหลืออยู่ด้วย $1/0.5 = 2$
- **ผลลัพธ์:** ค่าคาดหวัง (Expected Value) ของ Output ตอนเทรน จะเท่ากับตอนเทสต์พอดี

$$E[\tilde{y}] = E\left[\frac{1}{p}xr\right] = \frac{1}{p}xE[r] = \frac{1}{p}xp = x$$

- **ข้อดี:** ตอนใช้งานจริง (Inference) เราไม่ต้องทำอะไรเลย แค่ Forward Pass ปกติ จบ!

---

3. ทฤษฎีเรขาคณิต (Geometric Mean of Exponential Models)

ทำไมการสุ่มตัดแล้วมาเฉลี่ยทีหลังถึงดี?

Srivastava และ Hinton พิสูจน์ให้เห็นว่า การใช้ Dropout มีค่าเท่ากับการหา **Geometric Mean (ค่าเฉลี่ยเรขาคณิต)** ของโมเดลย่อยๆ ทั้งหมด

- สมมติเรามี Network ย่อยที่เป็นไปได้ทั้งหมด 2^n รูปแบบ (จากการสุ่มปิด n neurons)
- ให้ $p_{model}(y|x)$ คือความน่าจะเป็นที่โมเดลย่อยแต่ละตัวทำนายออกมา
- การเทรนด้วย Dropout คือการพยายามทำให้โมเดลหลัก (Shared Weights) สามารถเป็นตัวแทนของ **ค่าเฉลี่ยเรขาคณิต** ของโมเดลย่อยเหล่านั้น:

$$P_{ensemble}(y|x) \approx \sqrt[2^n]{\prod_{i=1}^{2^n} P_i(y|x)}$$

ความหมาย: มันคือการหา "จุดร่วม" ที่ทุกโมเดลย่อยเห็นพ้องต้องกันมากที่สุด ซึ่งให้ผลลัพธ์ที่แม่นยำและเสถียรกว่า Arithmetic Mean (ค่าเฉลี่ยปกติ) ในมิติที่สูงมากๆ

---

4. ผลต่อ Gradient Descent (Stochastic Regularization)

ในมุมมองของการปรับจูน Weight Dropout ทำหน้าที่เหมือนการ **"ใส่ Noise เข้าไปใน Gradient"**

- ปกติ: $\Delta w = -\eta \frac{\partial L}{\partial w}$ (ทิศทางชัดเจนแน่นอน)

- Dropout: $\Delta w = -\eta \left(\frac{\partial L}{\partial w} + \text{Noise} \right)$

Noise นี้ดีอย่างไร?

Noise นี้ป้องกันไม่ให้ Optimizer ไปติดอยู่ใน **"หลุมที่แคบและลึก" (Sharp Minima)** ซึ่งเป็นจุดที่ Overfit ได้ง่าย (เพราะถ้าขยับ Input นิดเดียว Error จะพุ่งปรี๊ด)

แต่ Noise จะผลักให้ Optimizer วิ่งไปหา **"หลุมที่กว้างและแบน" (Flat Minima)** แทน ซึ่งเป็นจุดที่ Generalization ดีกว่า (เพราะแม้ Input จะเปลี่ยนไปบ้าง Error ก็ยังต่ำอยู่)

4.3 Early Stopping: เวลาคือตัวแปรควบคุมความซับซ้อน

เทคนิคสุดท้ายที่เรียบง่ายแต่ทรงพลังคือการจัดการกับ "เวลา" ตามที่วิเคราะห์ไว้ในงานวิจัยของ **Lutz Prechelt (1998)** เรื่อง *"Early Stopping - But When?"* งานวิจัยนี้ชี้ให้เห็นว่าจำนวนรอบของการเทรน (Epochs) ไม่ได้เป็นเพียงตัวบอกระยะเวลา แต่เป็น Hyperparameter ที่ควบคุมความซับซ้อนของโมเดลโดยตรง

ธรรมชาติของการเรียนรู้ (Learning Dynamics) นั้น โมเดลจะเริ่มเรียนรู้โครงสร้างหลักที่สำคัญของข้อมูล (General Patterns) ในช่วงแรกของการเทรน ซึ่งเป็นช่วงที่ค่า Loss ลดลงอย่างรวดเร็ว แต่เมื่อเวลาผ่านไปและโมเดลเริ่มเก็บรายละเอียดหลักครบถ้วนแล้ว มันจะเริ่มหันไปจดจำรายละเอียดเล็กๆ น้อยๆ ที่เป็นส่วนเกินหรือ Noise ของข้อมูล Training Set แทน จุดนี้เองที่ความรู้เริ่มกลายเป็นการท่องจำ

ดังนั้น วิธีการ **Early Stopping** จึงทำหน้าที่เหมือนกรรมการที่คอยเป่านกหวีดหมดเวลา โดยการเฝ้าสังเกตค่าความผิดพลาดใน Validation Set (Validation Loss) อย่างใกล้ชิด ทันทีที่ค่านี้หยุดลดลงและเริ่มดีดตัวสูงขึ้น (สัญญาณของการ Overfitting) เราจะสั่งหยุดการเทรนทันทีและย้อนกลับไปใช้ค่าพารามิเตอร์ ณ จุดที่ดีที่สุด การทำเช่นนี้เปรียบเสมือนการหยุดอ่านหนังสือก่อนที่สมองจะเริ่มล้าและสับสน ซึ่งเป็นการจำกัดความซับซ้อนของโมเดลที่มีประสิทธิภาพสูงโดยไม่ต้องแก้ไขสมการคณิตศาสตร์ใดๆ

---

5. ภาคปฏิบัติ (Implementation Guide)

ทฤษฎีแน่นแล้ว มาดูวิธีเขียนโค้ดจริงใน PyTorch กันครับ ว่าเราจะ "ใส่ยา" รักษา Overfitting ลงไปในโค้ดได้อย่างไร

5.1 การใส่ Dropout (ง่ายเหมือนปลอกกล้วย)

ใน PyTorch, Dropout ถูกมองว่าเป็น "Layer" หนึ่งครับ เราแค่วางมันคั่นไว้ระหว่าง Hidden Layers

Python

```
import torch.nn as nn

class RegularizedModel(nn.Module):
    def __init__(self):
        super().__init__()
        # สมมติ Input มี 20 features
        self.layer1 = nn.Linear(20, 64)
        self.relu = nn.ReLU()

        # ใส่ Dropout ตรงนี้!
        # p=0.5 แปลว่ามีความน่าจะเป็น 50% ที่ Neuron ในชั้นนี้จะถูกปิดเป็น 0
        self.dropout = nn.Dropout(p=0.5)

        self.layer2 = nn.Linear(64, 1) # Output Layer

    def forward(self, x):
        x = self.layer1(x)
        x = self.relu(x)

        # Dropout จะทำงานเฉพาะตอน model.train() เท่านั้น
        # พอตอน model.eval() มันจะหยุดทำงานอัตโนมัติ (เปิดครบทุกตัว)
        x = self.dropout(x)

        x = self.layer2(x)
        return x
```

5.2 การใส่ L2 Regularization (Weight Decay)

อันนี้พิเศษหน่อยครับ เราไม่ได้ใส่ในตัวโมเดล แต่เราใส่ตอนประกาศ **Optimizer**

Python

```
import torch.optim as optim

# weight_decay คือพารามิเตอร์สำหรับ L2 Regularization
# ค่าปกติที่นิยมใช้คือ 1e-5 ถึง 1e-3
optimizer = optim.Adam(model.parameters(), lr=0.001, weight_decay=1e-5)
```

เคล็ดลับ:

- ถ้าโมเดลยัง Overfit อยู่: ให้เพิ่มค่า p ใน Dropout (เช่น จาก 0.2 -> 0.5) หรือเพิ่ม weight_decay
- ถ้าโมเดล Underfit (เรียนไม่รู้เรื่อง): ให้ลดค่าพวกนี้ลง หรือเอาออกไปเลย